

5. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    printf("\nOriginal Input:\n%s\n", input);

    printf("\nParsing quotes...\n");

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\'' ) {
            printf("Single quote found at position %d\n", i);
        }
        else if (input[i] == '"' ) {
            printf("Double quote found at position %d\n", i);
        }
    }

    return 0;
}
```

```
sharanaya@DESKTOP-TPK9V3Q gcc quotes.c -o quotes
sharanaya@DESKTOP-TPK9V3Q ./quotes
Enter command: echo 'Hello World' "Welcome Student"

Original Input:
echo 'Hello World' "Welcome Student"

Parsing quotes...
Single quote found at position 5
Single quote found at position 17
Double quote found at position 19
Double quote found at position 35
```

6.

To Create Process Escape Sequences, Handle

Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser

Output, Test Complex Inputs.

To Create Child Processes, Execute Programs, Handle Execution Errors,
Pass Arguments, Manage Parent Process, Test Command Launching.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char input[100], *args[10];
    int n = 0, i = 0;
    pid_t pid;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    // Parse command
    char *token = strtok(input, " ");
    while (token != NULL && n < 9) {
        args[n++] = token;
        token = strtok(NULL, " ");
    }
    args[n] = NULL;

    printf("\nParsed Arguments:\n");
    for (i = 0; i < n; i++)
        printf("Arg%d: %s\n", i + 1, args[i]);

    // Create child
    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        // Child
        execvp(args[0], args);
        perror("execvp");
        exit(1);
    }
    else {
        // Parent
        waitpid(pid, NULL, 0);
        printf("\nChild process completed.\n");
    }

    return 0;
}
```

```

sharanaya@DESKTOP-TPK9V3Q gcc Skill6.c -o Skill6
sharanaya@DESKTOP-TPK9V3Q ./Skill6
Enter command: echo Hello World

Parsed Arguments:
Arg1: echo
Arg2: Hello
Arg3: World
Hello World

Child process completed.

```

7. Process Synchronization and Child Process Monitoring Using waitpid().

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char cmd[50], path[1000], *dir;
    pid_t pid;

    printf("Enter command: ");
    scanf("%49s", cmd);

    strcpy(path, getenv("PATH"));
    dir = strtok(path, ":");

    while (dir != NULL) {
        char file[200];
        sprintf(file, "%s/%s", dir, cmd);

        if (access(file, X_OK) == 0) {
            printf("Executable found: %s\n", file);

            pid = fork();

            if (pid == 0) {
                execl(file, cmd, NULL);
                perror("Execution failed");
                exit(1);
            }
            else if (pid > 0) {
                waitpid(pid, NULL, 0);
                printf("Child process completed.\n");
            }
            else {
                perror("Fork failed");
            }

            return 0;
        }

        dir = strtok(NULL, ":");
    }

    printf("Command not found.\n");
    return 0;
}

```

```
sharanaya@DESKTOP-TPK9V3Q nano Skill7.1.c
sharanaya@DESKTOP-TPK9V3Q gcc Skill7.1.c -o Skill7.1
sharanaya@DESKTOP-TPK9V3Q ./Skill7.1
Enter command: ls
Executable found: /usr/bin/ls
Execution failed: Bad address
Child process completed.
*** stack smashing detected ***: terminated
Aborted ./Skill7.1
```